Post Pruning (or Cost Complexity Pruning)

Watch Cost Complexity Pruning first.

Summary : (with the help of deepseek)
———————

Tree = The tree created from the x_train dataset.
Pruned_Tree = The tree after pruning its specific subtree.
A subtree is a node and all its descendants, which also forms a tree. So a subtree has at least one leaf node.

$$\text{CCP}(\alpha) \text{ for a subtree} = \frac{\text{Error(Pruned_Tree)} - \text{Error(Tree)}}{\text{Leaves(Tree)} - \text{Leaves(Pruned_Tree)}}$$

Sklearn quantifies CCP on a dataset for each subtree by :

```
tree = DecisionTreeClassifier(max_depth = None)
tree.fit(X = x_train, y = y_train)
path = tree.cost_complexity_pruning_path(X = x_train, y = y_train) //
quantifing CCPs on training dataset.
ccp = ccp_path.ccp_alphas
```

So, Error(Tree) is the error that the Decision Tree makes on training dataset and Full Grown Decision Trees don't make any error during training. Error(Tree) = 0 here.

$$\text{Finally, CCP}(\alpha) \text{ for a subtree} = \frac{\text{Error(Pruned_Tree)} - \text{Error(Tree)}}{\text{Leaves(Tree)} - \text{Leaves(Pruned_Tree)}}$$

Leaves(Tree) is constant, assume its 20. Error(Pruned_tree) is almost 0.3 for the below changes of Leaves(Pruned_Tree).

Now for Leaves(Pruned_Tree) = 10, CCP = 0.3 / 10 = 0.03
for Leaves(Pruned_Tree) = 15, CCP = 0.3 / 5 = 0.06

The more the leaves get pruned, the higher the CCP. So the lower the CCP, the lower number of leaves get pruned from a tree and the higher the CCP, the higher number of leaves get pruned.

As you can see, CCP(α) represents

i) The average error a Pruned Tree makes per pruned|removed leaf.

ii) Cost | Added_Error per pruned|removed leaf.

At the beginning, a full grown decision tree(that we make on training input)'s all leaves(which represents the output) represents 100% correct output for the training dataset. So the CCP is 0 which means no leaf node makes any mistake on training data for that tree and the accuracy is 100% on training dataset which is Overfitting.
But if we prune a subtree from that tree, the CCP becomes positive decimal value because the pruned|removed leaves nodes used to tell us some outputs are not anymore in the full grown tree and adding some "error" in the accuracy since the tree is now smaller alongside reducing the "complexity" of overfitting. So as you can see that by pruning subtree the overfitting issue is reducing i.e. the accuracy on training dataset will decrease but increases on testing dataset.

Now at aboveee, CCP = 0.03 meant :

- Pruning 10 leaves caused the error to increase from 0 (full tree) to 0.3 (pruned tree).
- This means each pruned leaf contributed ~0.03 to the increase in error.
- It tells you the price paid in accuracy for reducing complexity by one leaf.

So finally what CCP(α) means?

- α quantifies the trade-off: A high α means each pruned leaf costs a lot of accuracy.
- Pruning decisions: We prune subtrees where α is small, not smallest (i.e., that pruning that gives simplicity with minimal accuracy loss).

Finding the best CCP_ALPHA value of a tree

For every decision tree, there is a ccp_alpha value. For a full grown tree, it's 0. For a tree whose specific branches are trimmed later have also a ccp_alpha value too. We're not simultaneously cutting different subtrees of a tree, NO! One CCP_ALPHA for a whole tree is quantified by trimming one subtree.

Now after trimming a left subtree from a tree, it has an accuracy on testing dataset. If we trim a right subtree, it also has an accuracy on testing dataset. So we've to check for which ccp_alpha value we're getting the highest accuracy score on the testing dataset. Why not on training dataset? Because our main goal is to end the overfitting issue for which obviously accuracy on training dataset will be lower but higher in testing dataset which is our goal, right?
So for each ccp_alpha value of a tree, the one that gives the highest accuracy score on testing dataset is the CCP_ALPHA we're seeking so that we can trim that subtree which returned that CCP_VALUE for which we got the highest accuracy on testing dataset.

Now when we prune the branches that don't contribute much to have a greater significance to classify the data? After the tree grows first fully, right?
In Sklearn's `DecisionTreeClassifier()`, there is a parameter '**ccp_alpha**'. We provide this value and after sklearn creates the entire tree, it prunes that subtree that has that ccp_alpha and hence some node values get changed to represent the new tree correctly.

The above code we saw at the beginning :

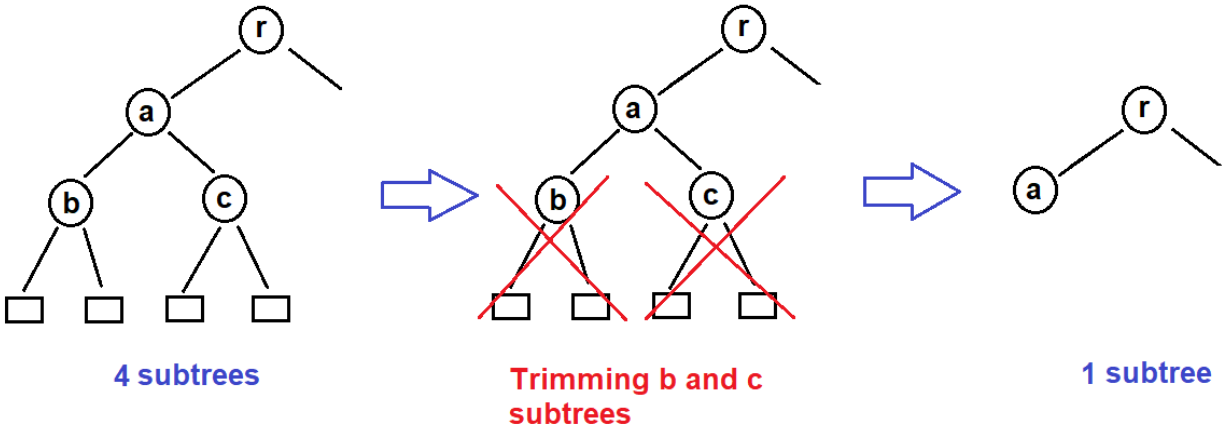
```
tree = DecisionTreeClassifier(max_depth = None)
tree.fit(X = x_train, y = y_train)
path = tree.cost_complexity_pruning_path(X = x_train, y = y_train) // quantifing CCPs on training dataset.
ccp = ccp_path.ccp_alphas
```

The output of ccp :

———————

ccp = array([0.00000000, 0.0021808, 0.0029304, 0.00431133, 0.00559513,
0.00743872, 0.0086638, 0.0129198, 0.01438561, 0.02404453,
0.02442056, 0.0306549, 0.3228506])

This ccp is quantified from a tree which has 16 Non Leaf Nodes. But the ccp has only 13 ccp_alpha values. Shouldn't there be 16 ccp_alpha values i.e. one ccp_alpha value for each subtree? For example :



At the left we can clearly see there are 4 subtrees(r, a ,b, c), so 4 ccp_alpha values. But after we pruned b and c subtrees, node a is no more a subtree but a mere leaf node. The only subtree we've at the right is node r. So technically we've 3 ccp_alpha values here but sklearn also returns the ccp_value 0 (which is "if we don't prune any subtree i.e. a full grown tree") because I think for smaller dataset, maybe the full grown tree which has only 2 - 4 depths is the best tree which returns highest accuracy score on testing dataset.
So for bigger tree, there are some nodes which is Subtree before BUT after the pruning starts those subtrees becomes leaf nodes and hence we don't get non_leaf_nodes_number_ccpAlpha_values which happened above where ccp has 13 ccp_alpha values but Non leaf Nodes number is 16.

Now to quantify the best CCP_ALPHA : For each ccp_value in ccp, we'll create a tree with that ccp_value and quantify the accuracy score for that tree and compare that accuracy with other trees.